

ECE 501 Project Week 2 Report

Project Title: Vehicle Orientation Estimation Using Conventional Image Processing

Group Number: 14

Project number: 8

Team Members:

Het Patel	AU2440221	het.p6@ahduni.edu.in
Shlok Shah	AU2440178	shlok.m2@ahduni.edu.in
Malay Panchal	AU2440050	malay.p@ahduni.edu.in
Dev Prajapati	AU2440022	dev.p12@ahduni.edu.in

Github Repository Link: https://github.com/hetpatel5107/DIP_2026_Project8_Group14

- **Introduction**

This week, we focused on understanding the DRASHTI-HaOBB dataset and its annotation format. Instead of estimating the vehicle orientation directly from the image pixels, we first used the information already provided by the dataset, such as the vehicle corner coordinates and heading angle. Using these annotations, we drew the bounding boxes and heading arrows on the original images, cropped individual vehicles, and explored how the heading angles can be represented as orientation values. This helped us understand the dataset and prepare the data for the actual orientation estimation step in the next phase.

- **Downloading and understanding label format**

After downloading and unzipping the test images and test labels ZIPs and uploading a small subset directly into Colab, we opened one label file and printed its contents:

Code:

```
with open("/content/0bGO_UHlP_zpWU.txt") as f:

    print(f.read())
```

Output:

```
... flightHeight:47.09
1707 774 2022 871 2062 739 1747 642 Sedan 0 17.1
1243 1081 993 1075 990 1196 1240 1202 SUV 0 181.38
863 654 999 658 1000 607 864 603 MotorCycle 0 1.68
1567 834 1684 919 1716 875 1599 790 MotorCycle 0 36.32
1472 1053 1344 1045 1341 1092 1469 1100 MotorCycle 0 183.62
1503 1595 1503 1291 1367 1291 1367 1595 Sedan 0 270.0
1891 1133 1642 1124 1638 1250 1887 1259 SUV 0 181.88
1298 926 1384 1007 1423 967 1337 886 MotorCycle 0 43.68
2219 934 1867 946 1872 1073 2224 1061 Pickup 0 178.04
3759 1903 3697 1904 3705 2026 3767 2025 MotorCycle 0 270.0
3815 1970 3764 1971 3777 2111 3829 2109 MotorCycle 0 270.0
3573 2002 3532 2003 3538 2112 3579 2111 MotorCycle 0 270.0
3454 1939 3348 1946 3389 2151 3495 2145 SUV 0 250.0
2418 429 2670 429 2670 300 2418 300 Van 0 0.0
1659 1955 1690 2130 1799 2111 1768 1936 AutoRicksaw 0 79.9
2247 2069 2131 2046 2108 2159 2229 2159 Van 1 101.4
1764 1698 1789 1816 1844 1804 1819 1686 MotorCycle 0 77.9
1902 1497 1981 1324 1883 1280 1805 1453 AutoRicksaw 0 294.3
1217 320 1417 320 1417 202 1217 202 AutoRicksaw 0 0.0
3747 872 2935 908 2943 1099 3755 1062 Tanker 0 177.4
```

Corresponding image to 0bGO_UHIP_zpWU.txt file:



Explanation:

The first line gives the drone's flight height in meters, here for the image we are using currently it is 47.09 . Each following line describes one vehicle: the first 8 numbers are the (x,y) pixel coordinates of the 4 corners of a rotated box, followed by the class name, a 0/1 difficulty flag, where 1 indicates that the vehicle is cropped at the image boundary and is only partially visible, and the heading angle in degrees.

- **Drawing the Box and Heading Arrow**

Code:

```
import math
from PIL import Image, ImageDraw

img = Image.open("0bGO_UH1P_zpWU.jpg").convert("RGB")
draw = ImageDraw.Draw(img)

with open("0bGO_UH1P_zpWU.txt") as f:
    lines = f.readlines()

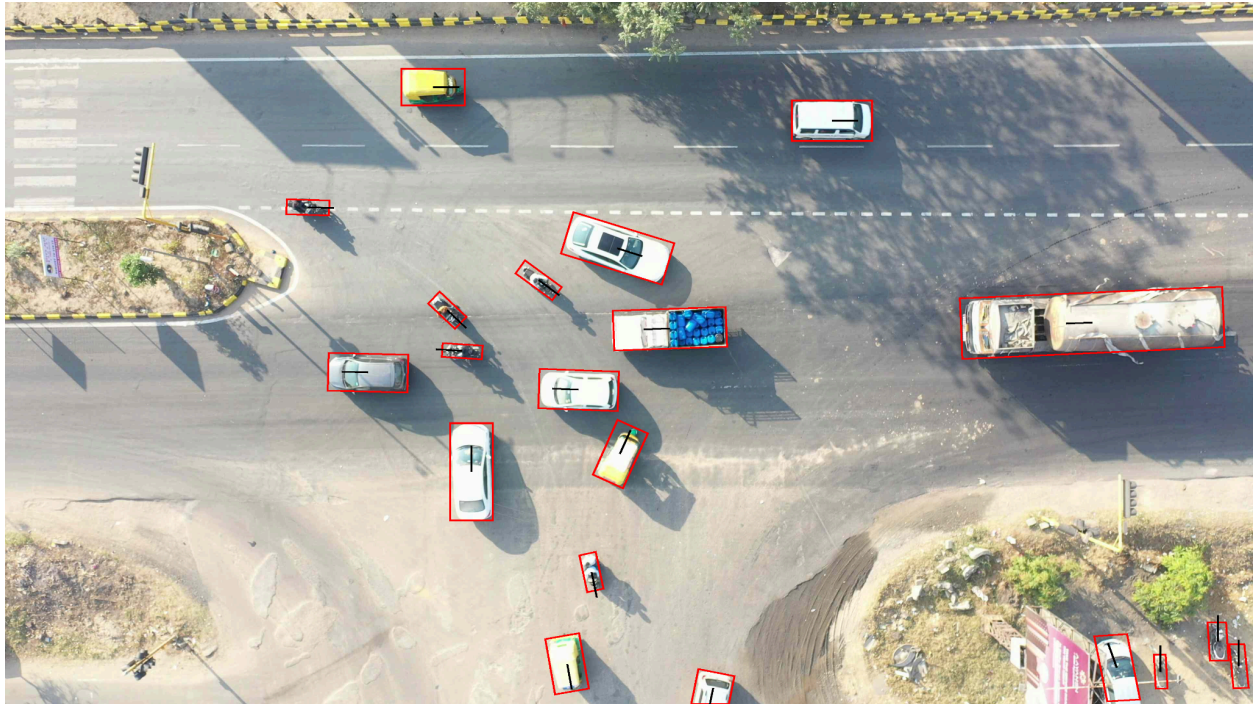
for line in lines[1:]:
    parts = line.split()
    coords = list(map(float, parts[0:8]))
    points = [(coords[i], coords[i+1]) for i in range(0, 8, 2)]
    cls = parts[8]
    heading = float(parts[10])

    draw.polygon(points, outline=(255, 0, 0), width=6)

    cx = sum(p[0] for p in points) / 4
    cy = sum(p[1] for p in points) / 4
    ax = cx + 80 * math.cos(math.radians(heading))
    ay = cy + 80 * math.sin(math.radians(heading))
    draw.line([(cx, cy), (ax, ay)], fill=(0, 0, 0), width=6)

img.save("boxed_output.jpg")
img
```

Output:



Explanation:

To verify our label parsing logic, we wrote a Python script using the Pillow (PIL) library to overlay the dataset coordinates directly onto the aerial image. The script skips the first metadata line of the label file, loops through each vehicle row, and extracts the first eight numbers to map out the four corner coordinates. It connects these points using `draw.polygon` to form a red bounding box around each vehicle. To show the direction of travel, the code averages those corner points to find the vehicle's center point (cx , cy) and uses basic trigonometry (sine and cosine) to project a black heading line 80 pixels out in the direction of the ground-truth angle. Looking at the output image, the red boxes trace the vehicles and the black arrows show their heading direction, which helped us confirm that the data was being extracted correctly and that we could move on to the next phase.

- **Cropping Individual Vehicles**

Code:

```
from PIL import Image
import os

img = Image.open("0bGO_UH1P_zpWU.jpg").convert("RGB")
W, H = img.size

with open("0bGO_UH1P_zpWU.txt") as f:
    lines = f.readlines()

os.makedirs("crops", exist_ok=True)
PAD = 8

for i, line in enumerate(lines[1:], start=1):
    parts = line.split()
    coords = list(map(float, parts[0:8]))
    xs = coords[0:2]
    ys = coords[1:2]
    x_min, x_max = max(0, min(xs)-PAD), min(W, max(xs)+PAD)
    y_min, y_max = max(0, min(ys)-PAD), min(H, max(ys)+PAD)

    crop = img.crop((x_min, y_min, x_max, y_max))
    cls = parts[8]
    heading = parts[10]
    crop.save(f"crops/v{i}_{cls}_{heading}deg.jpg")

print("Saved", len(lines)-1, "vehicle crops")
```

Some cropped images as output:



Explanation:

Basically, this code takes the original road image and the annotation file. The annotation gives us the four corner points of each vehicle. We use those points to find the vehicle's boundary, add a little padding, and crop it from the original image. Then we save each vehicle separately, along with its class and heading angle in the filename. So this step basically converts one large image into individual vehicle images that we can process further.

- **Converting Heading Angle to Orientation**

```
with open("0bGO_UHlP_zpWU.txt") as f:
    lines = f.readlines()

for line in lines[1:]:
    parts = line.split()
    cls = parts[8]
    heading = float(parts[10])
    orientation = round(heading % 90, 2)
    print(f"Class: {cls:<12} Heading: {heading:<8} -> Orientation: {orientation}")
```

Output:

```
.. Class: Sedan      Heading: 17.1    -> Orientation: 17.1
   Class: SUV        Heading: 181.38   -> Orientation: 1.38
   Class: Motorcycle Heading: 1.68     -> Orientation: 1.68
   Class: Motorcycle Heading: 36.32    -> Orientation: 36.32
   Class: Motorcycle Heading: 183.62   -> Orientation: 3.62
   Class: Sedan      Heading: 270.0     -> Orientation: 0.0
   Class: SUV        Heading: 181.88   -> Orientation: 1.88
   Class: Motorcycle Heading: 43.68    -> Orientation: 43.68
   Class: Pickup     Heading: 178.04   -> Orientation: 88.04
   Class: Motorcycle Heading: 270.0     -> Orientation: 0.0
   Class: Motorcycle Heading: 270.0     -> Orientation: 0.0
   Class: Motorcycle Heading: 270.0     -> Orientation: 0.0
   Class: SUV        Heading: 250.0     -> Orientation: 70.0
   Class: Van        Heading: 0.0       -> Orientation: 0.0
   Class: AutoRicksaw Heading: 79.9      -> Orientation: 79.9
   Class: Van        Heading: 101.4     -> Orientation: 11.4
   Class: Motorcycle Heading: 77.9      -> Orientation: 77.9
   Class: AutoRicksaw Heading: 294.3    -> Orientation: 24.3
   Class: AutoRicksaw Heading: 0.0      -> Orientation: 0.0
   Class: Tanker     Heading: 177.4     -> Orientation: 87.4
```

Explanation:

In this step, we take the heading angle given in the annotation file and convert it into a simpler orientation value. The heading angle can be anywhere from 0° to 360°, but for our analysis, we

are considering the orientation within a 90° range. So, we use the modulo % 90 operation to convert the angle into the range of 0° to 90° .

For example, a heading of 181.38° becomes 1.38° , while a heading of 270° becomes 0° . The code also prints the vehicle class, original heading, and converted orientation for each vehicle. This helps us check how the heading angles are being converted before using them for further processing.

1. Work done this week

- Downloaded the official DRASHTI-HaOBB test-split images and labels from Zenodo (DOI: 10.5281/zenodo.18278989).
- Read and correctly explained the real label file format, including the flightHeight:<value> line format.
- Drew oriented bounding boxes and heading-angle arrows on all 20 vehicles in a real sample image and visually confirmed they line up correctly.
- Cropped all 20 vehicles into individual image files, named by class and heading angle.
- Converted all 20 real heading angles (0-359 degrees) to orientation values (0-90 degrees) directly from the label file, and explained why this range is sufficient.
- Corrected the Week 1 reference paper mistake and identified a better-fitting reference (see Literature Review).

2. Work to be done next week

- Run this same pipeline (box drawing, cropping, angle conversion) across the full 5-image sample set, not just one image.
- Begin the actual orientation-estimation step: take a cropped vehicle image with its label hidden, and estimate its angle purely from the pixels using a conventional image-processing technique (edge detection, minimum-area rotated rectangle fit, or a Radon-transform-based method as in Liu et al. [2]).
- Compare the estimated angle against the real ground-truth angle already extracted this week, and measure the error.

3. Literature Review

In Week 1, we selected Tan and Baker [1] as our primary reference. On closer reading, this paper turned out not to fit our problem: it recovers a vehicle's full 3D pose (position and orientation) from an oblique, ground-level traffic camera using a ground-plane constraint and 3D vehicle models. Our problem is different which is a single, already-cropped, top-down (nadir) image where only a 2D rotation angle needs to be estimated, with no oblique perspective or 3D geometry involved. We are therefore replacing it with a more directly relevant reference.

Liu et al. [2] address vehicle detection and orientation estimation directly in aerial, top-down imagery — the same viewing geometry as the DRASHTI-HaOBB dataset. Their pipeline generates oriented bounding box proposals for vehicles and uses a Radon transform, a

classical image-processing technique closely related to the Hough transform, to determine each vehicle's principal orientation from its silhouette. This is a much closer structural match to our problem: estimating a 2D rotation angle of a rectangular object seen from directly overhead, using conventional image processing rather than 3D scene geometry.

References

[1] T. N. Tan and K. D. Baker, "Efficient image gradient based vehicle localisation," *IEEE Transactions on Image Processing*, vol. 9, no. 8, pp. 1343–1356, Aug. 2000. DOI: 10.1109/83.855430.

[2] C. Liu, Y. Ding, M. Zhu, J. Xiu, M. Li, and Q. Li, "Vehicle Detection in Aerial Images Using a Fast Oriented Region Search and the Vector of Locally Aggregated Descriptors," *Sensors*, vol. 19, no. 15, p. 3294, 2019. DOI: 10.3390/s19153294.

Link to the reference paper: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6695642/>

4. Challenges faced

- The test images ZIP alone is about 1.2 GB, too large to work with, so we worked with a small subset of images.
- The first line of the label file is formatted as flightHeight:<value> rather than a bare number, which required a small correction to our initial understanding of the format. Raw floating-point modulo arithmetic produced messy results (e.g. 1.3799999999999955), which we fixed by rounding the orientation value to 2 decimal places.
- Realizing partway through Week 2 that the Week 1 reference paper did not match our problem's geometry, which required re-searching the literature and updating the reference mid-project.